Coding Task (Week 6-8)
Group 5

- Maulana Anugra Putra (5025251159)
- I Gusti Agung Candra Nugraha (5025251169)
- Hussein Mohammad Mahsun (5025251170)

TSP Simulator

1. Problem Formulation and Modeling

The objective of this task is to design an efficient route for a Central Kitchen / Service Provider (SPPG) to deliver Makan Bergizi Gratis (MBG) mealboxes to all designated schools within an adjustable service radius x km, returning to the depot upon completion.

The problem is structured as a two-layer graph architecture:

1. **Underlying Road Network Digraph ($G_{road} = (V_{road}, E_{road})$):**
    - Downloaded from OpenStreetMap (OSM) for the city of Surabaya.
    - It is a **directed graph (digraph)** where directed edges account for traffic rules, including one-way streets, dual carriageways, and access restrictions.
    - Each edge $(u, v) \in E_{road}$ has a weight representing street distance in meters: $w(u, v) = length$.
2. **Complete Metric TSP Digraph ($G_{tsp} = (V_{tsp}, E_{tsp})$):**
    - The vertex set $V_{tsp} = \{v_0, v_1, v_2, \ldots, v_k$ consists of the SPPG depot v_0 and the k schools located within radius x km.
    - G_{tsp} is a complete directed graph where every pair (v_i, v_j) has a directed edge whose cost $C(v_i, v_j)$ equals the shortest path distance from v_i to v_j traversing G_{road}.
    - Because G_{road} is a digraph, edge costs are asymmetric in general: $C(v_i, v_j) \neq C(v_j, v_i)$.

2. Algorithmic Idea

Solving the Travelling Salesman Problem exactly on an arbitrary graph is NP-hard. For practical interactive delivery routing where k dynamically scales (e.g., $k \approx 20 -- 50$ schools), an exact brute-force search ($O(k!)$) or dynamic programming Held-Karp approach ($O(k^2 2^k)$) becomes computationally intractable.

To achieve real-time responsiveness (< 50 ms) while guaranteeing high-quality routing, a **two-phase hybrid heuristic approach** is utilized:

Phase 1: Distance Matrix Computation (Dijkstra's Algorithm)

Before solving the order of visit, an all-pairs shortest path matrix between all targets in V_{tsp} is constructed. For each node $v_i \in V_{tsp}$, single-source shortest path searches using **Dijkstra's Algorithm** with a min-priority queue are executed on G_{road} to populate $C(v_i, v_j)$ for all $v_j \in V_{tsp}$.

Phase 2: Tour Construction via Nearest Neighbor Heuristic

A greedy initial tour starting at the SPPG depot (v_0) is established:

1. Start at the current node $u = v_0$. Initialize the tour: $T = [v_0]$.
2. Identify the unvisited node $v_i \in V_{tsp} \setminus T$ that minimizes $C(u, v)$.
3. Append v to T, set $u = v$, and repeat until all k schools are visited.
4. Append v_0 to close the cycle back to the SPPG.

Phase 3: Tour Optimization via 2-Opt Local Search

The greedy Nearest Neighbor tour often contains sub-optimal crossings and twists. The **2-Opt heuristic** iteratively inspects all edge pairs (v_i, v_{i+1}) and (v_j, v_{j+1}) and performs a 2-edge exchange:

- It reverses the visiting sequence between stop $i + 1$ and stop j to determine if the alternative connection yields a shorter overall cycle cost.
- Swaps are accepted whenever $\Delta Cost < 0$.
- The algorithm terminates when a full iteration passes with no cost reduction, indicating that a local minimum has been reached.

Phase 4: Path Reconstruction

Because the actual road network is a digraph, the vehicle travels over physical streets. The ordered TSP tour sequence is expanded back into individual street segments on G_{road} using shortest-path node sequences. Overlapping street segments are traversed as necessary, satisfying the requirement that physical road segments may be revisited.

3. Implementation Details

The implementation is written in Python and organized into the following modules:

1. **download_data.py:**
    - Fetches the drivable road network of Surabaya using osmnx.graph_from_place("Surabaya, East Java, Indonesia", network_type="drive").
    - Retrieves points of interest tagged amenity=school via the Overpass API.
    - Computes polygon centroids for multi-node school campuses to guarantee single point coordinate representations.
    - Caches data into cache/surabaya_road.graphml and cache/schools_surabaya.geojson to eliminate redundant network overhead.
2. **app.py:**
    - **Spatial Filtering:** Computes the Haversine distance between the selected SPPG coordinate (lat_0, lon_0) and all school locations to filter points where $d \leq xkm$.

- **Node Mapping:** Projects geographic coordinates to the nearest network node in G_{road} using a spatial KD-tree lookup (scikit-learn).
- **Shortest Path Computation:** Uses networkx.shortest_path_length with edge attribute weight='length' to populate the cost matrix. Unconnected nodes are assigned an infinite penalty value (10^9).
- **2-Opt Solver:** Reorders the visiting stops and records solver execution time via time.perf_counter().
- **Visualization & Streamlit UI:** Uses Folium and streamlit-folium to render the interactive map, marking the depot, school pins, delivery radius boundary circle, and polyline paths traced over the road network.

4. Computational Complexity Analysis

Let:

- V_{road} = Number of vertices in the Surabaya road graph ($\approx 36,618$ nodes).
- E_{road} = Number of directed edges in the Surabaya road graph ($\approx 89,025$ edges).
- k = Number of delivery stops (1 SPPG depot + filtered schools within radius x).
- m = Number of 2-Opt improvement iterations until local convergence.

Pipeline Component	**Dominant Algorithm**	**Time Complexity**	**Space Complexity**
Haversine Distance Filter	Vectorized Great-Circle formula	$O(N_{total_schools})$	$O(N_{total_schools})$
Nearest Node Projection	KD-Tree Spatial Lookup	$O(k\,.\,log\,\lvert V_{road}\rvert)$	$O(\lvert V_{road}\rvert)$
Distance Matrix Construction	Dijkstra's Algorithm (k runs)	$O(k\,.\,(\lvert E_{road}\rvert + \lvert V_{road}\rvert log\,\lvert V_{road}\rvert))$	$O(k^2 + \lvert V_{road}\rvert)$
TSP Initial Tour	Nearest Neighbor	$O(k^2)$	$O(k)$

	Greedy Search		
TSP Route Refinement	2-Opt Local Search	$O(m \cdot k^2)$	$O(k)$
Route Geometry Reconstruction	Concatenated Shortest Paths	$O(k\,.\,(\|E_{road}\| + \|V_{road}\| log \|V_{road}\|))$	$O(\|V_{road}\|)$

Analysis Summary:

1. **Time Complexity:**
   The theoretical bottleneck is the pairwise shortest path search across the street digraph:
   $O_{total} = O(k\,.\,(\|E_{road}\| + \|V_{road}\| log \|V_{road}\|) + m\,.\,k^2)$
   Because $k << \|V_{road}\|$ search completes within fractions of a second (typically $< 30ms$ for $k \leq 50$), enabling rapid interactive adjustment during live evaluation.
2. **Space Complexity:**
   The memory footprint is dominated by the storage of the city-wide road network graph:
   $S_{total} = O(V_{road}\| + \|E_{road}\|) + k^2)$
   The distance matrix takes only $O(k^2)$ floating-point entries, consuming less than a few megabytes of RAM.

CPP Simulator

1. Problem Formulation and Modeling

The objective of this task is to design an efficient route for a DLH (Dinas Lingkungan Hidup) dump truck that collects public garbage from every major road within an adjustable service radius x km of a chosen base, returning to the base when finished.

The problem is structured as a two-layer graph architecture:

Underlying Road Network Digraph (G_road = (V_road, E_road)):

- Downloaded from OpenStreetMap (OSM) for the city of Surabaya through osmnx.
- Only roads tagged highway = trunk, primary, secondary or tertiary are kept.
- It is a directed graph (digraph): junctions are vertices, and each directed edge is a drivable road piece. A two-way street becomes two opposite edges, and a one-way street becomes a single edge, so traffic rules are respected.
- Each edge (u, v) in E_road has a weight equal to its street length in metres: w(u, v) = length.

Service Graph (G_svc = (V_svc, E_svc)), a subgraph of G_road:

- The depot is snapped to the nearest junction v0 of G_road.
- V_svc contains every junction whose Haversine distance to the base is at most x km. E_svc contains every edge whose two endpoints are both in V_svc.
- Only the strongly connected component that contains v0 is kept. Every edge in it can be reached from the base and the truck can get back to the base afterwards. Edges outside it are ignored.

The Directed Chinese Postman Problem (CPP):

Because G_svc is a digraph, an edge may only be driven in its own direction. Some edges therefore have to be driven again (deadhead) to reach unvisited roads, which is why the optimal walk is usually longer than the sum of all edge lengths.

2. Algorithmic Idea

Unlike the Travelling Salesman Problem, the directed Chinese Postman Problem is solvable exactly in polynomial time, so no heuristic or approximation is needed and the route found is the shortest possible. The key fact is that a connected digraph has a closed walk that uses every edge exactly once (an Eulerian circuit) if and only if every vertex has equal in-degree and out-degree. Real road networks violate this because of one-way streets, so the algorithm fixes the imbalance as cheaply as possible and then traces the circuit.

Phase 1: Build the Service Graph

The Haversine distance from the base to every junction is computed in one vectorised step. The nearest junction becomes v0. Junctions within x km are kept, the subgraph is formed, and only the strongly connected component of v0 is retained (Tarjan's algorithm).

Phase 2: Measure the Imbalance*

For each vertex v the imbalance is d(v) = out-degree(v) minus in-degree(v). Vertices with d(v) < 0 have too many arrivals and need extra departures (sources). Vertices with d(v) > 0 have too many

departures and need extra arrivals (sinks). If every d(v) is 0, the graph is already Eulerian and no edges are repeated.

Phase 3: Cheapest Matching of Sources to Sinks

1. Dijkstra's algorithm with a min-priority queue runs from each source on G_svc to get the shortest path length to every sink.
2. A min-cost flow problem is built: each source supplies |d(v)| units, each sink demands d(v) units, and sending one unit from source s to sink t costs the shortest-path length from s to t.
3. The network simplex solver in networkx finds the minimum-cost way to send all supply to demand. Costs are rounded to decimetres (integers) so the result is exact.

Phase 4: Duplicate the Repeated Edges

Every shortest path chosen in Phase 3 is expanded into its edges, and those edges are added to the graph as extra parallel copies. Each vertex now has equal in-degree and out-degree, and the added length is the minimum possible.

Phase 5: Eulerian Circuit (Hierholzer's Algorithm)

A closed walk that uses every edge, original or duplicate, exactly once is traced starting from v0. It is the optimal CPP tour.

Phase 6: Route Reconstruction and Time Estimate

Each hop of the circuit is turned back into a street segment with its name, geometry and speed. The first time an edge is driven it is labelled Collect, and any further time it is labelled Repeat (deadhead). Consecutive hops on the same street are merged into one line of the traversal table. Travel time is the sum of length divided by speed, where speed is the OSM maxspeed if present, otherwise a typical speed for the road class (trunk 50, primary 40, secondary 35, tertiary 30 km/h), multiplied by a user-set speed factor that models a slow collection truck.

Pseudocode

```
solve_cpp(G_road, lat, lon, x):
   v0, G = service_graph(G_road, lat, lon, x)     # Phase 1
   for v in G: d[v] = out_deg(v) - in_deg(v)      # Phase 2
   S = {v : d[v] < 0};  T = {v : d[v] > 0}
   for s in S: dist[s], path[s] = dijkstra(G, s)  # Phase 3
   flow = min_cost_flow(S, T, cost = dist)
   for (s, t, f) in flow:                         # Phase 4
      add f copies of every edge on path[s][t]
   return eulerian_circuit(G, start = v0)         # Phase 5
```

3. Implementation Details

The implementation is written in Python and organised into the following modules:

network.py

- Fetches the Surabaya road network with osmnx.graph_from_place, using a custom filter that keeps only trunk, primary, secondary and tertiary roads and their _link ramps. The result is a directed multigraph, so one-way streets stay one-way.
- Caches the download in data/surabaya_roads.graphml so later runs load instantly.
- Spatial Filtering: computes the Haversine distance between the chosen base (lat0, lon0) and every junction with vectorised numpy, then keeps junctions where d is at most x km.
- Node Mapping: snaps the base to the nearest junction with an argmin over those distances, and reports the snap distance.
- Reachability: keeps only the strongly connected component containing the base (networkx), and reports how many edges were dropped.
- Includes an offline synthetic grid with one-way and two-way streets for testing without internet.

cpp_solver.py

- Counts the in-degree and out-degree imbalance of every junction.
- Shortest Path Computation: runs networkx single-source Dijkstra on edge attribute weight = length from each source junction.
- Solves the source-to-sink matching with networkx.min_cost_flow, then duplicates the edges along the chosen shortest paths.
- Extracts the Eulerian circuit with networkx.eulerian_circuit starting at the base and labels each hop Collect or Repeat.
- Records solver execution time with time.perf_counter().

route.py

- Attaches street name, road class, geometry and speed to each hop of the circuit.
- Merges consecutive hops on the same street into runs for the traversal table.
- Computes travel time and builds the table with cumulative distance and elapsed minutes.

app.py

- Streamlit UI with Folium (via streamlit-folium). The user clicks the map or types coordinates to set the base, and adjusts the radius x and a speed factor with sliders.
- Draws the base marker, the radius circle and the route over the road network. Colour runs from blue (start) to red (end), and dashed lines are repeated segments.
- Displays total distance, estimated travel time, CPP solver execution time, repeated distance, the street traversal order table, and CSV and HTML downloads.

test_solver.py

- Checks that the route starts and ends at the base, is contiguous, covers every edge, and matches a hand-computed optimum. It is run on 30 random strongly connected digraphs and on the demo grid.

requirements.txt lists osmnx, networkx, folium, streamlit, streamlit-folium, pandas, numpy and shapely.

4. Computational Complexity Analysis

Let:

- |V_road|, |E_road| = number of vertices and directed edges in the filtered Surabaya road graph.
- |V|, |E| = number of vertices and directed edges in the service graph (inside radius x, in the base's strongly connected component), so |V| is at most |V_road|.
- s, t = number of source and sink junctions (unbalanced vertices), and U = s + t.
- F = total imbalance, the sum of |d(v)| over sources, and A = number of edge copies added (repeated segments).

Pipeline Component	Dominant Algorithm	Time Complexity	Space Complexity
Haversine Distance Filter and Nearest Node	Vectorised great-circle formula, argmin	O(|V_road|)	O(|V_road|)
Service Graph and Reachability	Subgraph extraction, Tarjan strongly connected components	O(|V| + |E|)	O(|V| + |E|)
Imbalance Computation	Degree count per vertex	O(|V| + |E|)	O(|V|)
Shortest Paths from Sources	Dijkstra's Algorithm (s runs)	O(s . (|E| + |V| log |V|))	O(|V| + s . t)
Source-to-Sink Matching	Min-cost flow (network simplex) on an s by t bipartite graph	Polynomial in U (no tight bound used here)	O(s . t)
Edge Duplication	Expand chosen shortest paths	O(A), with A at most F . |V|	O(|E| + A)
Eulerian Circuit	Hierholzer's Algorithm	O(|E| + A)	O(|E| + A)
Route Reconstruction	Attach names, geometry, speeds; merge runs	O(|E| + A)	O(|E| + A)

Analysis Summary

Time Complexity: The theoretical bottleneck is the repeated shortest-path search from every unbalanced junction, so the total is about O(|V_road| + s . (|E| + |V| log |V|) + matching). In road networks s is much smaller than |V| (only junctions where one-way streets meet unevenly are unbalanced), and the matching works on only s by t pairs, so solving is fast. On the offline demo grid in testing (200 directed edges, 1.5 km radius) the solver finished in about 9 ms and produced a 72.7 km route. Unlike the TSP, the result is exactly optimal, not a local minimum, because the directed CPP is polynomial.

Space Complexity: Memory is dominated by storing the road graph, O(|V_road| + |E_road|), plus the service graph and the added edge copies, O(|E| + A). The source-to-sink cost table needs only O(s . t) entries.

Scaling with the radius: Increasing x grows |V| and |E| roughly with the covered area, and so grows the number of unbalanced junctions s. Solver time therefore rises with x but stays polynomial.

Open item: add the measured solver time for your Surabaya test cases once you have run them.

Knight's Tour Simulator

1. Problem Formulation and Modeling

The objective of this task is to simulate the generation of a Knight's Tour on a dynamic chessboard. Given a board of `cols × rows` squares and a starting square, the program must find a sequence of knight moves that visits every square exactly once. If no such sequence exists, the program prints `path not found`.

Input

- Line 1: `cols rows` (size of the chessboard)
- Line 2: `start_x start_y` (starting position of the knight, 0-indexed)

Output

- The coordinates `x,y` of every square in visiting order, or `path not found`.

The problem is modeled as a graph problem:

1. **Knight Graph G = (V, E)**
    - Every square of the board is a vertex, so |V| = N = cols × rows.
    - Two vertices (x1, y1) and (x2, y2) are connected by an undirected edge if a knight can jump between them, i.e. the difference (dx, dy) is one of the 8 L-shaped moves: (±1, ±2) or (±2, ±1).
    - Each vertex has degree between 2 (corner squares) and 8 (squares far enough from every border).
2. **Tour as a Hamiltonian Path**
    - A Knight's Tour that starts at square s and visits every square exactly once is a Hamiltonian path in G that begins at vertex s.
    - Finding a Hamiltonian path is NP-complete for general graphs, so a naive exhaustive search is not practical for larger boards.
3. **G is Bipartite**
    - A knight always changes square colour on every move, so the vertices split into two sets (squares with x+y even and squares with x+y odd) and every edge connects the two sets.
    - This property gives a cheap necessary condition for the existence of a tour (see Phase 1 below).

2. Algorithmic Idea

To be both **correct** (it never misses an existing tour and never reports a false tour) and **fast** (practical for boards such as 8×8 or 20×20), a hybrid approach is used: exact backtracking guided by a heuristic, protected by a parity pre-check.

Phase 1: Feasibility Check (Parity / Colouring Argument)

Because G is bipartite, the vertices of a path alternate between the two colour classes. If N is odd, one class has exactly one more square than the other (the class containing the corner (0,0), where x+y is even). A path covering all N squares must therefore start on this larger class.

- If N is odd and (start_x + start_y) is odd, the program immediately prints `path not found`.
- This avoids an exponentially long exhaustive search on boards that provably have no tour from the chosen start.
- A start square outside the board is also rejected as `path not found`.

Phase 2: Depth-First Search with Backtracking

The search starts at the given square and builds the path one move at a time:

1. Mark the current square as visited and append it to the path.
2. If the path contains N squares, a tour has been found.
3. Otherwise, collect the unvisited squares reachable by a knight move and try them one by one, recursing into each.
4. If a branch reaches a dead end (no unvisited neighbour, but squares remain), undo the last move (unmark the square and remove it from the path) and try the next candidate.
5. If every candidate has been tried and failed, report that no tour exists from this position.

Because the search is exhaustive, `path not found` is only printed when no tour exists (or the parity check proved it impossible), so the result is exact.

Phase 3: Move Ordering with Warnsdorff's Heuristic

Plain backtracking would be far too slow on an 8×8 board. To avoid dead ends early, candidate moves are sorted before they are tried:

- **Warnsdorff's rule:** try first the candidate square that has the **fewest** unvisited onward moves. This forces the knight to visit hard-to-reach squares (borders and corners) before they become isolated and impossible to reach.
- **Tie-breaking:** when two candidates have the same number of onward moves, the one farther from the centre of the board (Manhattan distance) is tried first, which pushes the knight to finish the outer ring early.

Warnsdorff's rule alone is only a heuristic and can fail on some boards or starts. Combined with backtracking it keeps the speed of the heuristic while the backtracking guarantees completeness.

Pseudocode

```
solve(cols, rows, sx, sy):
  N = cols * rows
  if N is odd and (sx + sy) is odd: return NOT_FOUND
  visited = all False; path = []
  return backtrack(sx, sy)
```

```
backtrack(x, y):
    visited[x][y] = True; path.append((x, y))
    if len(path) == N: return SUCCESS
    C = unvisited knight-neighbours of (x, y)
    sort C by (number of unvisited neighbours ascending, distance from centre descending)
    for each (nx, ny) in C:
        if backtrack(nx, ny) == SUCCESS: return SUCCESS
    visited[x][y] = False; path.pop()      # undo the move
    return FAIL
```

Phase 4: Output

If a tour is found, the coordinates are printed in visiting order, followed by a board in which each square shows the step number at which it was visited (1 = starting square). This makes the result easy to verify by eye.

3. Implementation Details

The implementation is written in Python as a single module, `knights_tour.py`, organised as follows:

1. **Constants and data structures**
    - `MOVES`: the list of the 8 knight offsets (dx, dy).
    - `visited`: a `rows × cols` boolean matrix marking squares already on the path.
    - `path`: a list of (x, y) tuples holding the current partial tour.
2. **`solve_knights_tour(cols, rows, start_x, start_y)`**
    - Performs the parity check (Phase 1).
    - Defines the helper functions `on_board`, `free_neighbours` and the recursive `backtrack`.
    - Returns the full path as a list of coordinates, or `None` if no tour exists.
3. **`free_neighbours(x, y)`**
    - Applies the 8 offsets to (x, y) and keeps only the squares that are on the board and not yet visited. It is used both to generate candidate moves and to compute the Warnsdorff degree of each candidate.
4. **`backtrack(x, y)`**
    - Recursive DFS described in Phase 2, with candidates sorted by the key `(len(free_neighbours(candidate)), -distance_from_centre)` (Phase 3).
    - The recursion limit is raised with `sys.setrecursionlimit` because the recursion depth can reach N on large boards.
5. **`print_board(path, cols, rows)`**
    - Builds a grid of visit-order numbers and prints it, right-aligned for readability.

6. `main()`
    - Reads the two input lines, validates the start square, calls the solver and prints either the coordinates (one `x,y` per line) plus the numbered board, or `path not found`.

4. Computational Complexity Analysis

Let:

- N = cols × rows = the number of squares (vertices of the knight graph).
- Each vertex has at most 8 neighbours, so |E| ≤ 4N = O(N).
- Branching factor b ≤ 8 (at most 8 knight moves from any square).

Pipeline Component	Dominant Technique	Time Complexity	Space Complexity
Parity / bounds check	Colouring argument	O(1)	O(1)
Neighbour generation	Fixed set of 8 offsets	O(1) per call	O(1)
Move ordering per node	Warnsdorff sort (≤ 8 candidates, each ≤ 8 checks)	O(1) per node (about 64 checks plus an 8-element sort)	O(1)
Tour search, typical successful case	DFS guided by Warnsdorff	O(N)	O(N)
Tour search, worst case	Exhaustive backtracking	O(8^N)	O(N)
Output of path and board	Linear scan	O(N)	O(N)

Analysis Summary:

1. **Time Complexity**

- Every expanded search node costs constant time, because a knight has at most 8 moves and the Warnsdorff degree of each candidate is also computed over at most 8 moves.
- When Warnsdorff's ordering leads the search straight to a tour with little or no backtracking, only about N nodes are expanded, so the running time is O(N). This is what we observed in testing: boards from 5×5 up to 20×20 finished almost instantly.
- In the worst case the search must explore many partial paths. The number of simple paths is bounded by O(8^N), so the worst-case time is exponential, which is expected because the Hamiltonian path problem is NP-complete. This happens mainly for boards that have no tour and that the parity check cannot rule out. Small impossible boards such as 3×3 and 4×4 are still solved instantly, but very large impossible boards could be slow.
- The parity check removes an entire class of impossible inputs (odd-sized boards with a start square of the wrong colour) in O(1).

2. **Space Complexity**
   - The `visited` matrix uses O(N) memory, the `path` list holds at most N coordinates, and the recursion stack has depth at most N, so the total is O(N).
   - Printing the numbered board needs another O(N) grid.